Backend refactoring — IcfesApp

Branch	`refactor` (created from `App_test`, commit `663dfaf`)
Last update	2026-10-30
Status	Phases 1 to 4 done · Phases 5 and 6 pending

This document records **every change** made during the refactoring: what changed, why, and how to use it. It is meant for a developer joining the project, so they can understand the structure without reading all the code.

> **Language convention:** code, comments, XML docs, logs and developer-facing exception messages are written in **English**. Messages returned by the API to end users (e.g. `"Credenciales inválidas."`) and seed content (subject names, plan descriptions, emails) stay in **Spanish**, because the app is used by Colombian students.

---

1. Project map

The solution uses a layered architecture. Each layer only knows the layers below it:

```
IcfesApp.API            → HTTP controllers. All inherit from ApiControllerBase.
   |
IcfesApp.Infrastructure → Service implementations (EF Core, Identity, JWT, email).
   |
IcfesApp.Application    → Service interfaces (IXxxService), DTOs, Result and mappers (ToDto).
   |
IcfesApp.Domain         → Entities, enums, constants and QuestionBuilder. Depends on nothing.
```

How to add a new feature:

1. Entity in `Domain/Entities` (if needed).
2. DTOs in `Application/<Module>/Dtos` and their mapper in `Application/<Module>/<Module>Mappings.cs`.
3. `IXxxService` interface in `Application/Common/Interfaces`, returning `Result` / `Result<T>`.
4. `XxxService` implementation in `Infrastructure/Content` (or `Infrastructure/Security` for auth) and its registration in `DependencyInjection.cs`.
5. Controller in `API/Controllers` that **inherits from** `ApiControllerBase`.

---

2. Inheritance, interfaces and patterns

2.1 Class hierarchies

```
ControllerBase (ASP.NET)
 └── ApiControllerBase                 CurrentUserId + Result → HTTP
      ├── SignInControllerBase         Common sign-in response (tokens / 2FA required)
      │    ├── AuthController          register, register-staff, login, google, session, refresh, revoke
      │    └── TwoFactorController     2fa/setup, 2fa/enable, 2fa/disable, 2fa/verify
      ├── PasswordController           forgot-password, reset-password, change-password
      ├── EmailConfirmationController  confirm-email, resend-confirmation
      └── QuestionsController, PracticeSessionsController, SubjectsController, InstitutionsController,
          InvitationCodesController, PlansController, SubscriptionsController, ReportsController

Result                                 Succeeded + Errors + ErrorType
 ├── Result<T>                         + Value
 └── AuthResult                        + tokens, user, 2FA

AuthEmailMessage (abstract)            Template Method: BuildHtmlBody()
 ├── EmailConfirmationMessage
 └── PasswordResetMessage
```

2.1.1 Proxies (same interface, wrapping the real service)

```
Controller ──► ISubjectService         = CachedSubjectService                    ──► SubjectService
Controller ──► IPlanService            = CachedPlanService                       ──► PlanService
Controller ──► IPracticeSessionService = SubscriptionCheckingPracticeSessionService ──► PracticeSessionService
```

2.2 Design patterns in use

Pattern	Where	Problem it solves
Builder	`Domain/Builders/QuestionBuilder.cs`	A question was assembled by hand in 3 places with different rules.
Factory	`Infrastructure/Identity/ApplicationUserFactory.cs`	`new ApplicationUser { ... }` was repeated in 6 places.
Strategy	`Infrastructure/Content/SubscriptionOwnership/`	`SubscriptionService` had `if Particular / if Institutional` in two methods.
Template Method	`Infrastructure/Email/Messages/`	Both auth emails had the same HTML structure written twice.
Proxy (protection)	`Infrastructure/Content/Proxies/SubscriptionCheckingPracticeSessionService.cs`	`HasActiveAccessAsync` existed but nothing enforced subscriptions.
Proxy (caching)	`Infrastructure/Content/Proxies/CachedSubjectService.cs`, `CachedPlanService.cs`	Subjects and plans were queried on every request although they rarely change.
Base class (inheritance)	`ApiControllerBase`, `SignInControllerBase`	`CurrentUserId` and error handling were copied into every controller.

2.3 Main interfaces

Interface	Implementation	Responsibility
`IRegistrationService`	`RegistrationService`	Student sign-up and staff creation
`ISessionService`	`SessionService`	Login, Google login, refresh, revoke, current user
`IPasswordService`	`PasswordService`	Forgot / reset / change password
`IEmailConfirmationService`	`EmailConfirmationService`	Confirm email, resend link
`ITwoFactorService`	`TwoFactorService`	2FA setup, enable, disable, verify login
`IAuthTokenIssuer` *(infrastructure)*	`AuthTokenIssuer`	Issue and revoke session tokens
`IAuthEmailSender` *(infrastructure)*	`AuthEmailSender`	Send auth emails, log failures
`ISubscriptionOwnerStrategy` *(infrastructure)*	`Particular…` / `Institutional…`	Whose subscription covers a user
`IQuestionService`, `IPracticeSessionService`, …	`QuestionService`, …	Content modules (unchanged contracts)

---

3. Change log

Phase 1 — Bugs and security

1.1 Students could see the correct answers 🔒

- **Before:** `GET /api/questions` only required being logged in, and `QuestionDto` includes `IsCorrect`.
- **Now:** the whole `QuestionsController` requires the `RequireTeacher` policy (Teacher or Admin). Students keep getting their questions through `/api/practice-sessions`, which never reveals the answer ahead of time.
- **Frontend impact:** none; only the admin panel uses that endpoint.

1.2 Creating or editing a question dropped the images 🐞

- **Before:** `ImageUrl`, `JustificationImageUrl` and each option's image were not saved on create/update, and updating deleted the option images. `QuestionDto` did not return them either.
- **Now:** requests accept `imageUrl`, `justificationImageUrl` and `options[].imageUrl`, and the DTOs return them.
- **Frontend impact:** compatible. To keep images when editing from the admin panel, the frontend must send back the `imageUrl` values it received.

1.3 Login HTTP status depended on the message text 🐞

- **Before:** `AuthController` searched for words like "*confirmar tu correo*" or "*bloqueada*" inside the message to choose 401/403/423.
- **Now:** `Result` has an `ErrorType` and `ApiControllerBase.ToErrorResponse` maps it:

ErrorType	HTTP
`Validation`	400
`Unauthorized`	401
`Forbidden`	403 (email not confirmed)
`NotFound`	404
`Locked`	423 (account locked)

1.4 `register-staff` handed the new user's tokens to the admin 🔒

- **Now:** it returns only the created user (`UserInfoDto`). The frontend did not read that response.

1.5 `/api/auth/session` returned incomplete data 🐞

- **Before:** empty `firstName` / `lastName` and no `schoolId` (built only from the JWT).
- **Now:** it reads the user from the database.

1.6 Demo data with a known password in every environment 🔒

- **Now:** `DataSeeder` (users `studentX@test.com` / `Test12345!`) only runs in `Development`. Roles and the default admin are still created everywhere.

1.7 Anyone could see inactive plans 🔒

- **Now:** `includeInactive` only takes effect for an Admin.

1.8 Missing practice session returned 400 instead of 404 🐞

- `POST /api/practice-sessions/{id}/answers` and `/finish` now return **404**.

1.9 Reports used a hard-coded role string

- `[Authorize(Roles = "Teacher,Admin")]` → `[Authorize(Policy = Policies.RequireTeacher)]`.

1.10 Malformed 2FA token caused a 500 error 🐞 *(found while testing phase 3)*

- **Before:** `POST /api/auth/2fa/verify` with an invalid token (e.g. `"abc"`) crashed with a 500: `SecurityTokenMalformedException` derives from `ArgumentException`, not from the `SecurityTokenException` that was being caught.
- **Now:** both exceptions are caught and the endpoint returns 401.
- **File:** `Infrastructure/Security/JwtTokenService.cs`

Phase 2 — Patterns and clean code

2.1 Builder pattern for questions (`QuestionBuilder`)

The **only** way to build a question. It validates the business rules: statement and justification required, at least 2 options, exactly 1 correct option, every option with text.

```
var questionBuilder = QuestionBuilder.ForSubject(subjectId)
    .WithStatement("What is 2 + 2?", statementImageUrl: null)
    .WithJustification("2 + 2 = 4")
    .WithDifficulty(QuestionDifficulty.Easy)
    .CreatedBy(teacherUserId)
    .AddOption("4", isCorrect: true)
    .AddOption("5", isCorrect: false);

var validationErrors = questionBuilder.Validate();   // empty list = valid
if (validationErrors.Count == 0)
{
    Question question = questionBuilder.Build();     // throws if invalid
}
```

Used by `QuestionService.CreateAsync`, `QuestionService.UpdateAsync` (through `BuildOptionsFor`) and `DataSeeder` (an invalid question in the JSON is skipped instead of breaking startup).

2.2 Controller base class (`ApiControllerBase`)

- `CurrentUserId`, `ToActionResult(Result)` (204), `ToActionResult(Result<T>)` (200 + value), `ToErrorResponse(Result)`.
- It replaced code copied into 7 controllers.

2.3 A single `Result` type

- `OperationResult` was removed and `AuthResult` now **inherits** from `Result`.

2.4 `ToDto()` mappers — one place per DTO

One static class of extension methods per module: `QuestionMappings`, `PracticeSessionMappings`, `SchoolMappings`, `SubjectMappings`, `PlanMappings`, `InvitationCodeMappings` (uses the entity's `IsUsable`) and `SubscriptionMappings` (uses the entity's `IsCurrentlyActive`).

2.5 Behaviour in the entities

- `Question.CorrectOption` (it was `Options.First(o => o.IsCorrect)` repeated in 4 places).
- `StudentAnswer.IsAnswered` and `StudentAnswer.RegisterSelectedOption(option)`.

2.6 Practice session performance

- Starting a session used to load **every** question of the subject with its options. Now it loads only the ids, picks randomly and loads just the selected ones.
- The owner filter is now part of the SQL query.

- The question seeder used one query per question (~230); now it uses one.
- Read-only queries use `AsNoTracking()`.

2.7 Clear variable names

One-letter lambdas (`q`, `s`, `o`, `a`, `u`, `ic`, `ps`, `ur`, `rt`) and generic names (`result`, `stored`, `key`, `item`, `opt`, `total`, `ex`, `db` ...) were replaced with descriptive names (`question`, `createUserResult`, `storedRefreshToken`, `authenticatorKey`, `questionSeed`, `totalQuestionCount`, `exception`, `dbContext` ...). Magic numbers became constants (`RandomSuffixLength`, `UnlimitedUses`, `RecoveryCodeCount`, `KeyGroupLength`). Noise comments like `// <--- AGREGADO AQUÍ (ESTE ERA EL ERROR)` were deleted.

Phase 3 — Splitting `AuthService`, Factory and Strategy

3.1 `AuthService` (542 lines, 7 responsibilities) split into focused services

New service	Methods	Previously in
`RegistrationService`	`RegisterStudentAsync`, `RegisterStaffAsync`	`AuthService.RegisterAsync`, `RegisterStaffAsync`
`SessionService`	`LoginAsync`, `LoginWithGoogleAsync`, `RefreshAsync`, `RevokeRefreshTokenAsync`, `GetCurrentUserAsync`	`AuthService`
`PasswordService`	`SendPasswordResetEmailAsync`, `ResetPasswordAsync`, `ChangePasswordAsync`	`AuthService.ForgotPasswordAsync`, ...
`EmailConfirmationService`	`ConfirmEmailAsync`, `ResendConfirmationEmailAsync`	`AuthService`
`TwoFactorService`	`GetSetupAsync`, `EnableAsync`, `DisableAsync`, `VerifyLoginAsync`	`AuthService.*TwoFactor*`
`AuthTokenIssuer`	`CompleteSignInAsync`, `IssueTokensAsync`, `RevokeAllRefreshTokensAsync`	private helpers of `AuthService`
`AuthEmailSender`	`SendEmailConfirmationAsync`, `SendPasswordResetAsync`	private helpers of `AuthService`

`IAuthService` was replaced by the five interfaces listed in section 2.3. Every class now has one reason to change.

3.2 `AuthController` split into 4 controllers — routes unchanged

`AuthController`, `PasswordController`, `EmailConfirmationController` and `TwoFactorController` share the `api/auth` prefix, so every URL is exactly the same as before. `SignInControllerBase` holds the sign-in response shared by `AuthController` (login, google) and `TwoFactorController` (verify).

3.3 Factory pattern for users (`ApplicationUserFactory`)

The only place that creates an `ApplicationUser`. Rules in one spot: `UserName` = email, the type is derived from the institution, and who has to confirm the email.

Method	Used by	Email confirmed?
`CreateSelfRegisteredStudent(email, first, last, institutionId?)`	`RegistrationService`	No
`CreateStaff(email, first, last)`	`RegistrationService`	Yes
`CreateFromGoogle(email, givenName?, familyName?)`	`SessionService`	Yes
`CreatePreconfirmed(email, first, last, institutionId?)`	`RoleSeeder`, `DataSeeder`	Yes

3.4 Strategy pattern for subscriptions (`ISubscriptionOwnerStrategy`)

"Whose subscription covers this user?" is now decided by a strategy:

Strategy	Applies when	Filters by
`InstitutionalSubscriptionOwnerStrategy`	Institutional user **with** an institution	`InstitutionId`
`ParticularSubscriptionOwnerStrategy`	Any other user	`UserId`

The conditions never overlap, so exactly one strategy applies to each user. `SubscriptionService.GetMineAsync` and `HasActiveAccessAsync` share it, with no `if` per user type. A new owner type = a new class registered in `DependencyInjection.cs` .

> **Small behaviour change:** an *institutional* user **without** an institution assigned is now treated as particular in `HasActiveAccessAsync` too (before, `GetMineAsync` did so but `HasActiveAccessAsync` returned `false`). Both methods are now consistent.

3.5 Template Method pattern for auth emails

`AuthEmailMessage` (abstract) defines the structure in `BuildHtmlBody()` : greeting → explanation → action link → "if it wasn't you" notice. `EmailConfirmationMessage` and `PasswordResetMessage` only override the texts. Email content is identical to before.

3.6 Dependency injection grouped by area

`DependencyInjection.cs` now has `AddAuthenticationServices` and `AddContentServices` .

3.7 Comments and developer texts translated to English

295 comment lines in 82 files, plus logs, configuration exceptions, the Swagger description and `.csproj` comments.

Phase 4 — Proxy pattern

A proxy implements **the same interface** as the real service and wraps it. Controllers keep asking for the interface and never know the proxy is there. Proxies are wired in `DependencyInjection.cs` with a small generic helper:

```
// Whoever asks for ISubjectService gets CachedSubjectService, which wraps the real SubjectService.
AddWithProxy<ISubjectService, SubjectService, CachedSubjectService>(services);
AddWithProxy<IPlanService, PlanService, CachedPlanService>(services);
AddWithProxy<IPracticeSessionService, PracticeSessionService, SubscriptionCheckingPracticeSessionService>(services)
```

To add a proxy to another service: create a class that implements the interface, receives the real service in its constructor, and register it with `AddWithProxy` .

4.1 Protection proxy — subscription required to practice

- **Class:** `SubscriptionCheckingPracticeSessionService` .
- **What it does:** before `StartAsync` it calls `ISubscriptionService.HasActiveAccessAsync` (which uses the phase 3 Strategy). Without an active subscription it returns **403** `"Necesitas una suscripción activa para iniciar una práctica."` .
- **What it does not block:** answering, finishing and viewing history, so a student whose subscription expires can still finish a session in progress. Teachers and admins are never blocked.
- **On/off switch** in `appsettings.json` (read live through `IOptionsMonitor` , no restart needed):

`json "PracticeAccess": { "RequireActiveSubscription": false }`

> ⚠️ **It is OFF by default on purpose.** The API has no way to create subscriptions yet (no payment flow, no automatic free trial), so turning it on would lock out every new particular user. Turn it on once that flow exists. - **Why a proxy:** `PracticeSessionService` knows nothing about subscriptions; the business rule lives in one place and can be switched off.

4.2 Caching proxy — subjects

- **Class:** `CachedSubjectService` (uses `IMemoryCache`, already included in the ASP.NET Core shared framework — nothing to install).
- `GetAllAsync` is cached for 30 minutes; `GetByIdAsync` is served from that same cached list.
- `CreateAsync`, `UpdateAsync` and `DeleteAsync` **clear the cache**, so the next read is always fresh.

4.3 Caching proxy — plans

- **Class:** `CachedPlanService`.
- One cache entry per filter combination (e.g. `plans:particular:active`), 10 minutes.
- There is no endpoint that edits plans (they are changed directly in the database), so entries just expire: a plan changed in the database shows up at most 10 minutes later.

> The cache lives in the memory of each API instance. If the API ever runs on several servers, switch to a distributed cache (e.g. Redis) behind the same proxies.

---

4. API behaviour changes (for the frontend)

Endpoint	Change	Breaks the frontend?
`/api/questions` (all methods)	Teacher/Admin only (403 for students)	No (admin panel only)
`GET/POST/PUT /api/questions`	New `imageUrl`, `justificationImageUrl`, `options[].imageUrl` fields	No
`POST /api/auth/register-staff`	Returns `UserInfoDto` instead of tokens	No (response not read)
`GET /api/auth/session`	Now includes `firstName`, `lastName`, `schoolId`	No
`POST /api/practice-sessions/{id}/answers` and `/finish`	404 when the session does not exist (was 400)	No
`GET /api/plans?includeInactive=true`	Ignored unless Admin	No
`POST /api/auth/2fa/verify` with malformed token	401 (was 500)	No
`POST /api/practice-sessions`	403 without an active subscription **only if** `PracticeAccess:RequireActiveSubscription` is `true` (default `false`)	No (off by default)
`GET /api/subjects`, `GET /api/plans`	Served from cache (same response)	No

No endpoint has been renamed yet (that is phase 5).

---

5. Testing

Build: **0 errors, 0 warnings.** Manual tests against the compiled API on a separate port (`5299`) with its own database (the developer's running API and database were not touched).

Phases 1–2

#	Test	Result
1	Login with wrong password	401 ✅
2	`/api/auth/session` as a student	Returns name, last name and `schoolId` ✅
3	Student calls `/api/questions`	403 ✅
4	Teacher calls `/api/questions`	234 questions, 70 with images ✅
5	Create question with no correct option	400 with the builder message ✅
6–8	Create (with images), update, delete question	Images saved; 204; 204 ✅
9–12	Start / answer / answer finished session / finish missing session	Correct; 400; 404 ✅
13–15	`register-staff` , inactive plans as anonymous, report, current subscription	✅

Phase 3

#	Test	Result
1–3	Login, wrong password, session	200 / 401 / 200 ✅
4–7	Refresh, reuse old refresh token, revoke, refresh revoked	New token / 401 / 204 / 401 ✅
8–9	Sign-up with invitation code, login before confirming	200 / 403 ✅
10–11	Duplicate sign-up, invalid invitation code	400 / 400 ✅
12–14	Forgot password, resend confirmation, bad confirmation token	200 / 200 / 400 ✅
15–16	Change password (wrong / right), login with the new password	400 / 200 / 200 ✅
17–19	2FA setup, enable with wrong code, verify with malformed token	200 / 400 / 401 ✅ *(was 500, fixed — see 1.10)*
20	Current subscription: institutional student, particular active, particular expired, teacher	INSTITUTIONAL / STANDARD / FREE (Expired) / INSTITUTIONAL ✅
21–22	`register-staff` valid / with role `Student`	200 / 400 ✅

Phase 4

#	Test	Result
1	Gate **on** — institutional student with active subscription starts practice	201 ✅
2	Gate **on** — particular student with active subscription	201 ✅
3	Gate **on** — particular student with expired subscription	403 with the message ✅
4	Gate **on** — teacher / admin without subscription	201 / 201 ✅
5	Gate **on** — expired student views their history	200 ✅
6	Gate **off** (default) — expired student starts practice	201 ✅
7	5 × `GET /api/subjects` + `GET /api/subjects/{id}` after the first load	0 SQL queries ✅
8	Create / update / delete subject, then read	New data visible immediately, deleted → 404 ✅
9	3 × `GET /api/plans?target=Particular`	1 SQL query ✅
10	Plans by filter and by id	3 particular / 4 total / by id ✅

> Emails are not actually delivered locally: Gmail rejects the SMTP credentials in `appsettings.json` (`5.7.8 Username and Password not accepted`). The API handles it as designed: it logs the error and still answers 200.

---

6. Pending

Phase	Content	Status
5	Rename endpoints to clear names and remove duplicates (requires frontend changes)	Pending
6	Rename `School` → `Institution` (with migration) · split `DataSeeder` per entity	Pending

Technical debt: `StudentAnswer.AnsweredAtUtc` gets the current date when the session is created, even before the question is answered. Making it nullable needs a migration, and the `dotnet-ef` tool is not installed on this machine (`dotnet tool install --global dotnet-ef`).